Foundations of Dynamic Programming Part I: Formalization, State Space Reduction, Combinatorial Proofs, and Implementation of 0/1 Knapsack, Subset Sum, Target Sum, Unbounded Knapsack, and Fractional Knapsack

Abstract

This paper presents a formal, unified algorithmic analysis of foundational Knapsack paradigms in Dynamic Programming (DP) derived from advanced computer science instruction. First, we provide an explicit reference guide detailing every asymptotic symbol, state-space vector, modular property, and pseudo-polynomial bound notation. Second, we establish the theoretical foundations of multi-dimensional state caching and contrast optimal substructure with overlapping subproblems. Third, we formalize the 0/1 Knapsack problem, proving its Bellman recurrence $dp[i][w] = \max(dp[i-1][w], v_i + dp[i-1][w-w_i])$ and demonstrating the backward-iteration space collapse theorem from $O(N \cdot W)$ to $O(W)$. Fourth, we analyze Subset Sum and Partition Equal Subset Sum, deriving boolean state transitions and proving $O(N \cdot S/64)$ bitset parallel acceleration. Fifth, we model the Target Sum problem, deriving a closed-form parity reduction $S_1 = (S_{total} + T)/2$. Sixth, we prove forward-iteration space optimization for Unbounded Knapsack ($x_i \in \mathbb{N}_0$). Seventh, we contrast discrete 0/1 Knapsack with Fractional Knapsack ($x_i \in [0, 1]$), proving greedy optimality under value density sorting $\rho_i = v_i/w_i$ and providing a mathematical counterexample for discrete failure. Production-grade, compilable Java implementations are provided for all discussed algorithms.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

1 Mathematical Symbols, Formal Notation, and Accessible Reference Guide

To maintain rigorous theoretical clarity and accessibility across all computational backgrounds, this section provides an explicit, self-contained reference guide for every mathematical symbol, dynamic programming state operator, set-theoretic notation, and asymptotic complexity bound utilized throughout this paper.

1.1 Asymptotic Complexity Notations

- Big-O Notation ($\mathcal{O}$ or O): Worst-Case Upper Bound
  - Formal Definition: $f(N) = O(g(N))$ if there exist positive constants c and N_0 such that $f(N) \leq c \cdot g(N)$ for all $N \geq N_0$.
  - Plain Explanation: Represents the guaranteed maximum consumption of time or space resources.
  - Context: The 0/1 Knapsack problem operates in $O(N \cdot W)$ pseudo-polynomial time, where N is the item count and W is the knapsack capacity.
- Big-Theta Notation (Θ): Exact Asymptotic Bound
  - Formal Definition: $f(N) = \Theta(g(N))$ if $f(N) = O(g(N))$ and $f(N) = \Omega(g(N))$.
  - Plain Explanation: Describes the precise growth rate of an algorithm when upper and lower bounds coincide.
- Pseudo-Polynomial Time Complexity:
  - Formal Definition: An algorithm whose running time is polynomial in the numeric value of the input (e.g., W), but exponential in the bit-length of the input ($\log W$).
  - Context: While 0/1 Knapsack is NP-Hard in the general sense, its dynamic programming formulation runs in $O(N \cdot W)$ time, which is polynomial when W is bounded.

TABLE 1
Mathematical Notation Quick Reference Guide

Symbol	Name / Meaning	Plain-Language Explanation
$O(g(N,W))$	Big-O Notation	Asymptotic upper bound on execution time or space.
$\Theta(g(N,W))$	Big-Theta Notation	Exact asymptotic complexity bound.
$dp[i][w]$	2D Dynamic State Table	Maximum value using first i items with capacity w.
$dp[w]$	1D Compressed State Vector	Maximum value achievable with exact capacity w.
w_i, v_i	Weight and Value of Item i	Attributes of the i-th candidate element.
W	Knapsack Capacity Bound	Upper constraint on total permissible weight.
$\rho_i = v_i/w_i$	Value Density Ratio	Profit per unit weight of item i.
$x_i \in \{0,1\}$	Discrete Inclusion Variable	Decision variable (0 = Exclude, 1 = Include).
$x_i \in [0,1]$	Continuous Inclusion Fraction	Fraction of item i selected (Fractional Knapsack).
$x_i \in \mathbb{N}_0$	Unbounded Selection Quantity	Integer quantity of item i selected $(0,1,2,\dots)$.
$\vee$	Logical OR Operator	Boolean state transition for feasibility problems.

2 Taxonomy and Theoretical Foundations of Knapsack Paradigms

The Knapsack family represents one of the most thoroughly studied optimization paradigms in theoretical computer science. Originating in operations research and combinatorial optimization, knapsack problems encapsulate resource allocation under hard capacity constraints.

2.1 Categorization of Knapsack Variants

Knapsack problems are classified along two primary dimensions: item divisibility and item supply multiplicity.

1) 0/1 Knapsack Problem (Discrete & Bounded Supply): Each item $i \in \{1,\dots,N\}$ is indivisible ($x_i \in \{0,1\}$) and available in at most one copy.
2) Bounded Knapsack Problem (Discrete & Multi-Copy Supply): Each item i is available in at most c_i copies ($x_i \in \{0,1,\dots,c_i\}$).
3) Unbounded Knapsack Problem (Discrete & Infinite Supply): Each item i is indivisible, but available in unlimited quantities ($x_i \in \mathbb{N}_0$).
4) Fractional Knapsack Problem (Continuous & Divisible): Items can be arbitrarily split into fractional quantities ($x_i \in [0,1]$).

2.2 Core Prerequisites for Dynamic Programming

For discrete knapsack problems, dynamic programming is applicable due to two fundamental properties:

Definition 1 (Optimal Substructure). An optimal solution to a knapsack problem with N items and capacity W contains within it optimal solutions to subproblems involving $N-1$ items and capacity $W - w_N$ (if item N is included) or capacity W (if item N is excluded).

Definition 2 (Overlapping Subproblems). The recursive decision tree branches into duplicate subproblem instances parameterised by the tuple (i,w), where multiple distinct selection choices lead to identical remaining capacity w at item index i.

3 Case Study I: The 0/1 Knapsack Problem

3.1 Formal Problem Definition

Given N items, where item i has weight $w_i \in \mathbb{Z}^+$ and value $v_i \in \mathbb{Z}^+$, and a total knapsack capacity $W \in \mathbb{Z}^+$, maximize total value:

$$\max \sum_{i=1}^{N} v_i x_i \quad \text{subject to} \quad \sum_{i=1}^{N} w_i x_i \le W, \quad x_i \in \{0,1\}$$

3.2 Bellman Recurrence Formulation

Let $dp[i][w]$ denote the maximum value achievable considering a subset of the first i items with total weight capacity w. For $1 \le i \le N$ and $0 \le w \le W$:

$$dp[i][w] = \begin{cases} dp[i-1][w] & \text{if } w_i > w \\ \max\Big(dp[i-1][w],\ v_i + dp[i-1][w-w_i]\Big) & \text{if } w_i \le w \end{cases}$$

with base conditions $dp[0][w] = 0$ for all $0 \le w \le W$ and $dp[i][0] = 0$ for all $0 \le i \le N$.

3.3 Space Optimization: 2D Table to 1D Vector

Theorem 3 (Backward Iteration Space Collapse for 0/1 Knapsack). The 2D dynamic programming table of size $(N+1) \times (W+1)$ can be compressed into a 1D vector of size $W+1$ by iterating the capacity parameter w backwards from W down to w_i.

Proof. In the 2D recurrence, computing row i depends exclusively on values from row $i-1$:

$$dp^{(i)}[w] = \max\left(dp^{(i-1)}[w],\ v_i + dp^{(i-1)}[w-w_i]\right)$$

If we update a single 1D array $dp[w]$ in-place, updating $dp[w]$ requires the value of $dp[w-w_i]$ from the previous step $i-1$. If we iterate w in decreasing order ($W \to w_i$), the entry at index $w-w_i$ has not yet been modified in step i, thus preserving its state from step $i-1$. This prevents an item from being included multiple times in the same step. □

4 Case Study II: Subset Sum and Partition Equal Subset Sum

4.1 Problem Reduction and Boolean Recurrence

The Subset Sum problem asks whether there exists a subset of non-negative integers $A = \{a_1, a_2, \ldots, a_N\}$ that sums to exactly S.

Defining $dp[j]$ as a boolean flag indicating whether sum j is achievable:

$$dp[j] = dp[j] \vee dp[j-a_i] \quad \text{for } j \text{ from } S \text{ down to } a_i$$

Partition Equal Subset Sum reduces to Subset Sum with target $S = \sum A/2$. If $\sum A$ is odd, the partition is impossible ($dp = \text{false}$).

4.2 Bitset Parallel Acceleration

Theorem 4 (Bitwise Word Parallelism for Subset Sum). Using bitset representation, Subset Sum transitions can be executed in $O(N \cdot S/w_{word})$ time complexity, where w_{word} is the machine word size (64 bits).

The bitset update rule is:

$$\text{bitset} = \text{bitset} \mid (\text{bitset} \ll a_i)$$

5 Case Study III: Target Sum Problem

5.1 Mathematical Transformation to Subset Sum

Given an array of integers $nums$ and a target T, assign $+1$ or -1 to each element such that the signed sum equals T.

Let P be the set of elements assigned a $+1$ sign, and N_{set} be the set assigned a -1 sign:

$$\sum P - \sum N_{set} = T$$

Adding $\sum P + \sum N_{set} = \sum nums = S_{total}$ to both sides:

$$2 \cdot \sum P = S_{total} + T \implies \sum P = \frac{S_{total} + T}{2}$$

Corollary 5 (Target Sum Parity and Bound Conditions). A valid sign assignment exists if and only if: 1. $S_{total} + T \geq 0$ 2. $(S_{total} + T) \pmod 2 = 0$ Under these conditions, Target Sum is mathematically identical to counting subsets that sum to $S_{target} = (S_{total} + T)/2$.

6 Case Study IV: Unbounded Knapsack and Coin Change

6.1 Recurrence and Forward Iteration Proof

In Unbounded Knapsack, item supply is infinite ($x_i \in \mathbb{N}_0$).

$$dp[w] = \max_{1 \leq i \leq N,\ w_i \leq w} \left(v_i + dp[w-w_i]\right)$$

Theorem 6 (Forward Iteration Space Collapse for Unbounded Knapsack). Unbounded Knapsack is solved in $O(W)$ space by iterating capacity w forwards from w_i up to W.

Proof. Forwards iteration allows $dp[w-w_i]$ to reflect updates made within the current item iteration. Thus, item i can be repeatedly selected at capacities $w-w_i, w, w+w_i$, correctly modeling infinite item supply. □

7 Case Study V: Fractional Knapsack — Greedy vs Dynamic Programming

7.1 Continuous Relaxation and Density Sorting

When items are divisible ($x_i \in [0, 1]$), the problem is solved greedily.

Theorem 7 (Greedy Choice Optimality for Fractional Knapsack). Sorting items by value density ratio $\rho_i = v_i/w_i$ in descending order and greedily filling capacity W yields a globally optimal solution in $O(N \log N)$ time.

Proof. Suppose an optimal solution O^* selects an item j with lower ratio $\rho_j < \rho_k$ instead of an available item k. Replacing a fraction ϵ of item j with item k increases total value by $\epsilon(\rho_k - \rho_j) > 0$, contradicting the assumption that O^* was optimal. □

Corollary 8 (Failure of Greedy Strategy for 0/1 Knapsack). Greedy density sorting fails for discrete 0/1 Knapsack because item indivisibility leaves unused capacity gaps, preventing lower-density items from being replaced by optimal combinations of higher-density items.

8 Production-Grade Java Implementations

This section provides complete, compilable Java reference implementations for all discussed algorithms.

8.1 0/1 Knapsack (2D Tabulation and 1D Space Optimized)

```
public class Knapsack01 {
    // 2D Tabulation Approach: O(N*W) Time, O(N*W) Space
    public static int solve2D(int[] wt, int[] val, int W) {
        int n = wt.length;
        int[][] dp = new int[n + 1][W + 1];

        for (int i = 1; i <= n; i++) {
            for (int w = 1; w <= W; w++) {
                if (wt[i - 1] <= w) {
                    dp[i][w] = Math.max(dp[i - 1][w], val[i - 1] + dp[i - 1][w - wt[i - 1]]);
                } else {
                    dp[i][w] = dp[i - 1][w];
                }
            }
        }
        return dp[n][W];
    }

    // 1D Space-Optimized Approach: O(N*W) Time, O(W) Space
    public static int solve1D(int[] wt, int[] val, int W) {
        int n = wt.length;
        int[] dp = new int[W + 1];

        for (int i = 0; i < n; i++) {
            for (int w = W; w >= wt[i]; w--) { // Backward iteration preserves 0/1 constraint
                dp[w] = Math.max(dp[w], val[i] + dp[w - wt[i]]);
            }
        }
        return dp[W];
    }
}
```

8.2 Subset Sum and Target Sum Solvers

```
public class SubsetAndTargetSum {
    // Subset Sum: O(N*S) Time, O(S) Space
    public static boolean isSubsetSum(int[] arr, int target) {
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;

        for (int num : arr) {
            for (int j = target; j >= num; j--) {
                dp[j] = dp[j] || dp[j - num];
```

```
            }
        }
        return dp[target];
    }

    // Target Sum: O(N*S) Time, O(S) Space
    public static int findTargetSumWays(int[] nums, int target) {
        int totalSum = 0;
        for (int num : nums) totalSum += num;

        if (totalSum + target < 0 || (totalSum + target) % 2 != 0) return 0;
        int s1 = (totalSum + target) / 2;

        int[] dp = new int[s1 + 1];
        dp[0] = 1;

        for (int num : nums) {
            for (int j = s1; j >= num; j--) {
                dp[j] += dp[j - num];
            }
        }
        return dp[s1];
    }
}
```

8.3 Unbounded Knapsack and Fractional Knapsack Solvers

```
import java.util.Arrays;
import java.util.Comparator;

public class UnboundedAndFractionalKnapsack {
    // Unbounded Knapsack: O(N*W) Time, O(W) Space
    public static int unboundedKnapsack(int[] wt, int[] val, int W) {
        int[] dp = new int[W + 1];

        for (int w = 1; w <= W; w++) {
            for (int i = 0; i < wt.length; i++) {
                if (wt[i] <= w) {
                    dp[w] = Math.max(dp[w], val[i] + dp[w - wt[i]]);
                }
            }
        }
        return dp[W];
    }

    // Fractional Knapsack (Greedy): O(N log N) Time, O(N) Space
    static class Item {
        int weight, value;
        Item(int w, int v) { this.weight = w; this.value = v; }
    }

    public static double getMaxValue(int[] wt, int[] val, int capacity) {
        Item[] items = new Item[wt.length];
        for (int i = 0; i < wt.length; i++) items[i] = new Item(wt[i], val[i]);

        Arrays.sort(items, (a, b) -> Double.compare(
            (double)b.value / b.weight, (double)a.value / a.weight));

        double totalValue = 0.0;
        int currentCapacity = capacity;
```

```
        for (Item item : items) {
            if (currentCapacity >= item.weight) {
                currentCapacity -= item.weight;
                totalValue += item.value;
            } else {
                totalValue += ((double)item.value / item.weight) * currentCapacity;
                break;
            }
        }
        return totalValue;
    }
}
```

9 Comparative Synthesis and Master Decision Taxonomy

TABLE 2
Master Comparative Taxonomy of Knapsack Algorithmic Paradigms

Algorithm Paradigm	Item Supply	Time Complexity	Space Complexity	Capacity Loop	Optimality Class
0/1 Knapsack (2D)	Discrete (≤ 1)	$O(N \cdot W)$	$O(N \cdot W)$	$1 \to W$	Exact DP
0/1 Knapsack (1D)	Discrete (≤ 1)	$O(N \cdot W)$	$O(W)$	$W \to w_i$	Exact DP
Subset Sum	Discrete (≤ 1)	$O(N \cdot S)$	$O(S)$	$S \to a_i$	Exact DP
Target Sum	Discrete (≤ 1)	$O(N \cdot S)$	$O(S)$	$S_1 \to num$	Exact DP
Unbounded Knapsack	Infinite ($\mathbb{N}_0$)	$O(N \cdot W)$	$O(W)$	$w_i \to W$	Exact DP
Fractional Knapsack	Continuous ($[0, 1]$)	$O(N \log N)$	$O(N)$	N/A	Greedy Optimal